Selecting Reliable Answers with Pairwise Ranking and Arithmetic Evidence

DASE7506 MP2 · Student ID 3036780955 · 26 October 2026

Abstract

Selecting among four mathematical solutions requires distinguishing a correct answer from a convincing explanation. We combine fixed solution embeddings, information about the other candidates, and explicit arithmetic checks in a linear ranker. Training on correct-versus-incorrect candidates from the same question aligns the model with the selection task, while strong regularization improves its validation accuracy. Arithmetic features supply an additional signal for detecting numerical contradictions. The final method selects 738 of 798 validation answers correctly and reaches **81.1563% accuracy on the 467-question test bank**. On an Apple M5, complete test prediction and independent scoring take **0.315 seconds**, with a **26,064-byte** checkpoint.

1. The problem: plausible solutions can agree and still be wrong

The task is to choose one of four existing solutions to each mathematics question. Each candidate comes with its text and a fixed 1,024-dimensional embedding. The official parser extracts its final numeric answer, and any candidate matching the reference earns the point. The bank contains 4,755 training, 798 validation, and 467 test questions. We also report the 355-question validation subset with at least two distinct valid answers, where the selector must resolve an actual disagreement. All test questions satisfy this condition.

Agreement is useful but incomplete evidence. Majority voting answers 245 of these 355 disagreement questions correctly. A wrong answer can be repeated by several candidates, and even a well-written explanation can contain a simple addition error. Our approach combines three sources of evidence: the embedding of the solution, its relation to the alternatives, and numerical consistency within the text.

2. A shared score combines relative and local evidence

Candidate representation. We first normalize each embedding to unit length. Four features describe the candidate's context: its parsed answer's frequency among the four candidates, log(1 + text length), and the mean and maximum cosine similarity to the other three embeddings. Four further features describe the arithmetic evidence: log(1 + number of checked equalities), log(1 + number of inconsistent equalities), the inconsistent fraction, and an indicator that no equality could be checked. Standardization uses training candidates only. The resulting vector has 1,032 dimensions.

Arithmetic checks. A deterministic parser extracts adjacent numerical expressions in explicit equality chains. It supports basic arithmetic, bounded integer powers, fractions, and common LaTeX wrappers. Exact rational arithmetic evaluates each side. A displayed decimal result receives a tolerance of half its last decimal place. Variables and ambiguous expressions, including percentage and quotient-with-remainder notation, remain unchecked. An unchecked expression and a contradictory equality produce different features.

Within-question training. Let P_q contain every pair of a correct candidate i and a valid incorrect candidate j for question q. With standardized features z and score s(z) = w^T z, the objective is:

```
L(w) = sum_q [mean_(i,j in P_q) log(1 + exp(-(s_i - s_j)))] + ||w||^2 / (2C).
```

Each informative question has total weight one. This gives 6,516 pairs from 2,114 training questions. We fit the final model with C = 0.01, L-BFGS, tolerance 1e-6, and at most 2,000 iterations. The seed is 17. At inference, the model chooses the highest-scoring candidate with a parseable final answer. Exact ties use the lowest index; if all answers are unparseable, it returns index 0. The exported predictor uses NumPy.

3. Ranking and regularization account for most of the validation improvement

The progression from independent correctness prediction to within-question ranking improves selection on disagreement questions. We first compare majority voting, logistic regression on embeddings, and logistic regression with the four context features. The two classifiers each use C in {0.1, 1, 10}. Pairwise training then compares candidates directly, using the same context features. Reducing C from 1 to 0.01 improves validation accuracy while substantially reducing the training-validation gap.

Method	Validation correct / 798	Validation accuracy	Disagreement correct / 355	Disagreement accuracy
Answer majority vote	688	86.22%	245	69.01%
Embedding classifier, C=0.1	718	89.97%	275	77.46%
Context classifier, C=10	724	90.73%	281	79.15%
Pairwise context ranker, C=1	730	91.48%	287	80.85%
Pairwise context ranker, C=0.01	735	92.11%	292	82.25%
Pairwise MLP, 32 hidden units	734	91.98%	291	81.97%
Pairwise ranker + arithmetic, C=0.01	**738**	**92.48%**	**295**	**83.10%**

Table 1. Validation results. Each classifier row shows its best C. All methods select correctly on the remaining 443 questions without distinct valid answers. The disagreement subset therefore exposes their differences more clearly.

Strong regularization is more useful here than adding a hidden layer. The C=1 ranker reaches 91.15% on disagreement training questions and 80.85% on disagreement validation questions. With C=0.01, these become 82.73% and 82.25%. A 32-unit tanh MLP reaches 96.64% training accuracy but 81.97% validation accuracy at its selected checkpoint. Its training accuracy continues rising after that point without improving validation accuracy. These results motivate retaining a strongly regularized linear scoring function.

Arithmetic evidence improves this strongest linear baseline by three validation questions. With the model, C, pair construction, and standardization protocol held fixed, adding the four features corrects five selections and reverses two previously correct ones. The disagreement accuracy rises from 82.25% to 83.10%. The final model's disagreement training accuracy is 83.44%.

The development budget comprised ten fitted models: six initial classifiers, two linear context rankers, one MLP, and one arithmetic-augmented ranker. The MLP used Adam with learning rate 0.001, batch size 64 questions, L2 coefficient 0.001, and validation checks every five epochs. It stopped at epoch 30 and selected epoch 10. Recorded optimizer-update time across the ten fits totals about 13.32 seconds on the Apple M5; preprocessing, validation, and parser development are recorded separately in the experiment logs. We used validation for these choices and evaluated the final selected version once on test.

4. Arithmetic evidence catches a specific class of selection errors

The training diagnostic shows that explicit contradictions are informative. The checker evaluates at least one equality in 14,602 of 19,020 candidates (76.77%), covering 35,314 equalities. It flags 331 candidates. Among candidates with a valid final answer, 264 flagged candidates are incorrect and 47 are correct. Equivalently, 6.95% of incorrect candidates and 0.33% of correct candidates are flagged. This motivates using the signal as a score feature while retaining the other evidence.

The parser was refined on training text before the validation comparison. An initial audit exposed currency delimiters, percentage conventions, and remainder notation that could be misread as inconsistent equalities. After correcting these cases, a stratified inspection of 20 flagged training candidates found 19 clear numerical inconsistencies and one implicit-rounding ambiguity. The implementation and audit records accompany the code.

5. The gains and remaining errors explain what the model has learned

The five recovered validation questions all contained a real arithmetic contradiction in the previously selected candidate. Three examples are shown below. These are failures that a fixed embedding-based score had accepted despite the calculation being directly checkable.

Question	Previously selected calculation	New choice
Snails eaten over five days	3 + 5 + 7 + 9 + 11 = 36	35
Tips over three days	56 + 168 + 72 = 300	296
Population before people move away	15 x 58,940 = 883,100	Uses 884,100, then selects 377,050

Table 2. Representative recovered selections. The other two recoveries involve a candy total and a bag-average calculation.

Correct arithmetic alone does not identify the quantity the question asks for. In one regression, the new model selects a shoe box's uncovered volume, 32 cubic inches, instead of its uncovered area, 8 square inches. Both candidates have consistent arithmetic and identical arithmetic features. Their ordering changes through the jointly refitted embedding and context weights. This example points to the remaining problem of relating the solution's quantity and units to the question.

Parser coverage can also influence the score. In the other regression, the correct restaurant-bill solution includes the tip and reaches 61, but its currency and percentage formatting produces no checkable equalities. The selected solution reaches 56, omits the tip, and contains two consistent equalities. This illustrates why the amount of checkable content is an imperfect indicator of answer quality.

The scoring target also shapes how we interpret successful choices. In the recovered bag-average example, the newly selected candidate reaches the correct final answer, 8, through a flawed account of the bags. The selector improves the official final-answer outcome while that explanation still needs correction. Across the examples, arithmetic consistency complements the learned score; question interpretation and completeness remain important sources of error.

6. Final test result and resource use

The frozen method answers **379 of 467 test questions correctly, or 81.15631691648822%**. The earlier context classifier answered 366 questions correctly (78.372591%). The accumulated ranking, regularization, and arithmetic changes therefore add 13 correct selections, a gain of 2.78373 percentage points. The validation comparison in Table 1 isolates the final arithmetic-feature addition.

Measurement	Final model
Test correct / total	379 / 467
Official test accuracy	81.15631691648822%
Fresh prediction process	0.25330 s
Independent scoring process	0.06153 s
Complete prediction + scoring	0.31483 s
Checkpoint file	26,064 bytes
Uncompressed checkpoint entries	25,456 bytes
CPU and thread limit	Apple M5, 4 threads

Table 3. Timing includes process startup, imports, data and checkpoint loading, preprocessing, prediction, and scoring. These are local Apple M5 measurements; the course reference CPU is the Xeon Platinum 8457C. The supplied checkpoint is the complete inference asset, below the 16 MiB limit.

7. Reproduction, attribution, and conclusion

The submission package contains the frozen NumPy predictor, checkpoint, training code, experiment records, and the official test-score JSON. The teacher package is pinned to commit `bb801f4f2e7629d270df765ab12c333b2dc616bb`. Its parser and scoring programs are used unchanged. The README gives data restoration, environment setup, training, and evaluation commands. The final model uses only the supplied training candidates and references; validation guides model selection. Reproduction checks compare the retrained parameters with the supplied checkpoint and verify the validation result.

The method follows the verifier approach of scoring candidate solutions [1] and uses a pairwise logistic ranking objective [2]. Explicit calculation is inspired by the separation of language reasoning and program execution in PAL [3]. Here, the arithmetic component checks numerical equalities already present in the fixed candidates. Training uses scikit-learn and inference uses NumPy. Questions and references derive from GSM8K; the teacher generated candidates and representations with Qwen3-0.6B. The original data license and attribution notice are included.

AI assistance. Codex assisted with literature review, experiment planning, implementation, execution, error analysis, and report drafting. The submitted files include the resulting code, configurations, and measured outputs.

Conclusion. Within-question ranking and strong regularization provide the main selection gains. Adding explicit arithmetic evidence recovers a further three validation questions with a small inference cost. The resulting selector reaches 81.1563% test accuracy using a compact linear model. Its remaining mistakes center on what the question asks and which conditions the solution includes, providing a concrete distinction between checking calculations and understanding a solution.

References

[1] Karl Cobbe et al. *Training Verifiers to Solve Math Word Problems*. 2021. https://arxiv.org/abs/2110.14168

[2] Christopher Burges et al. *Learning to Rank using Gradient Descent*. ICML, 2005. https://www.microsoft.com/en-us/research/wp-content/uploads/2005/08/icml_ranking.pdf

[3] Luyu Gao et al. *PAL: Program-aided Language Models*. ICML, 2023. https://proceedings.mlr.press/v202/gao23f.html